

Daily Scholar Papers Report — 2026-08-30

2026-08-30

Daily Scholar Papers Report — 2026-08-30

[Download PDF](#)

Window covered: 2026-08-29 → 2026-08-30 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

Two alert threads, five candidates, three papers through screening — and the three of them turn out to be the same paper written three times, about three different systems.

Every automated testing technique that checks *behaviour* rather than *crashes* needs a specification someone has to write. Property-based testing needs properties. Mutation fuzzing needs seeds. The specification is where the bugs are found, and it is also the part that does not scale, because a human writes it. All three papers this window attack that bottleneck, and — this is the interesting part — none of them solves it by asking a language model to invent the specification from nothing. All three mine an artefact the organisation already owns, and use the model only to translate.

The clearest statement of the pattern comes from an ISSTA '26 experience paper on **property authoring for mobile apps**. Its thesis is that PBT frameworks have solved execution and input generation while assuming properties already exist, and that authoring them means learning a DSL *and* manually digging through view hierarchies for widget identifiers like `id="line"`. The fix is a two-phase pipeline whose first phase does no property synthesis at all: it annotates every widget with a semantic label and a functionality description, using a multimodal model fed the page screenshot with the target widget outlined in a red box. Only then does a second model write the code. The ablation is the whole argument — **95.0 % accuracy with the annotations, 76.9 % without**, an 18.1-point drop, reproduced at 88.2 % vs 71.7 % across 1,520 paraphrased descriptions. Grounding, not generation, is where the value sits.

An ASE '26 paper from the same lab takes the industrial half of the problem. **WeChat Pay** already runs a model-based acceptance-testing system over roughly 120 payment scenarios and 10,000+ business rules, and it already has a formal model of every use case — it just generates one test per page and has no oracle for any sequence outside that set. So the paper never asks anyone to write a property: it *extracts* them, keeping only transitions whose target page is provably unique across all explored paths, which is what makes them usable as oracles. **208 properties, 59 previously unknown bugs (48 logic, 11 crash), 55 confirmed and 45 fixed**, across 20,000 machine hours on a 2,000-device cluster. Neither the incumbent system nor classic PBT found any of them.

The third paper, at OOPSLA 2026, moves one layer down to **DBMS fuzzing** and makes the sharpest version of the argument. Its claim is not that existing fuzzers mutate badly — it is that when a SQL feature is absent from the seed corpus, insert/delete/replace mutation cannot synthesise it, so the corresponding paths are unreachable *regardless of fuzzing duration*. That is a reachability claim, not a

probability claim, and no reviewer can answer it with “run longer.” The seeds are built from two artefacts nobody was mining: official SQL reference documentation, and historical crash reports decomposed into reusable features. **61 previously unknown bugs across four DBMSs, 60 confirmed and fixed, one CVE**, and up to **12.5× branch coverage** over the host fuzzer’s default seeds.

Read together the rule is: *an LLM is most useful when it is translating an artefact you already have, and least useful when it is inventing one you do not*. Documentation, crash logs, acceptance-test models, view hierarchies — all three teams found their specification lying around unexploited, and the model’s job was transcription.

Two candidates were screened out at Stage-1 as saturated; they are not named.

Outstanding: 3 · Keep: 0 · Borderline High-Priority: 0

Highlighted Papers

Title	Authors	Venue	Link
From Natural Language to Executable Properties for Property-Based Testing of Mobile Apps (Experience Paper)	Y. Xiong, T. Su, J. Sun, J. Wang, Q. Li, G. Pu, Z. Su	<i>Proc. ACM Softw. Eng.</i> 3, ISSTA, Article ISSTA017, Oct 2026, 24 pp.	doi:10.1145/3832108 · author PDF · artifact
Model-Guided Property-Based Testing of WeChat Pay at Billion-User Scale	X. Shen, Y. Wang, T. Su, J. Liang, J. Sun, X. Liang, H. Sun, X. Xu, H. Lu, Y. Deng, P. Wang, G. Pu, Z. Su, J. Hughes	ASE ’26 — 41st IEEE/ACM Int. Conf. on Automated Software Engineering, Munich, Oct 2026, 13 pp.	doi:10.1145/3832783.3834537 · author PDF
SmartFuzz: Leveraging Large Language Models and Feature Composition to Generate High-Quality Seeds for Database Fuzzing	L. Lin, J. Hong, Y. Zhuang, R. Wu	<i>Proc. ACM Program. Lang.</i> 10, OOPSLA2, Article 344, Oct 2026, 29 pp.	doi:10.1145/3839476 · author PDF

Papers

1.1 · PROPERTY-BASED TESTING · Annotate the widgets first: 95.0% vs 76.9% accuracy, and k=0 demonstrations collapse all 124 outputs [👍🤔👎](#)

From Natural Language to Executable Properties for Property-Based Testing of Mobile Apps (Experience Paper)

Authors: Yiheng Xiong, Ting Su, Jingling Sun, Jue Wang, Qin Li, Geguang Pu, Zhendong Su

Venue: *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA017 (October 2026), 24 pages. Received 2026-01-30; accepted 2026-06-25.

Links. [ACM DOI 10.1145/3832108](https://doi.org/10.1145/3832108) · [author-hosted PDF](#) · [artifact](#). **License: CC BY 4.0** — the front matter states the work is licensed under a Creative Commons Attribution 4.0 International License. Full text read (24 pp.).

The bottleneck being named

Property-based testing for mobile apps works. Kea, PBFdroid, PDTDroid and ChimpCheck all execute properties and generate inputs competently. The paper’s observation is that they “primarily focus on the execution and input generation phases, assuming the existence of high-quality executable properties” — and that the assumption is doing enormous unacknowledged work, because *writing* a property is the unautomated stage.

Three barriers are named concretely. A tester must learn a framework-specific DSL, which is a steep curve for anyone without a strong programming background. A tester must then perform what amounts to view-hierarchy archaeology: locating low-level widget identifiers such as `id="Search"` by manual inspection. And a tester must implement a *complete* property under framework constraints, which is error-prone.

Classical rule-based natural-language-to-code approaches do not transfer, for two reasons the paper separates carefully: they lack flexibility, because enumerating rules for every paraphrase (“open the folder” vs “navigate into the directory”) does not scale and costs the manual effort you were trying to save; and they lack semantic grounding, because they cannot map a high-level description like “search bar” onto a low-level identifier. The authors then compress both into one sentence that licenses the entire architecture: the deeper challenge is **grounding the informal test intent into concrete executable properties**.

The running example is Amaze file manager. The property — clicking a directory should open it and display its contents, rather than triggering a file-opening dialog — is one sentence in English and eleven lines of Kea Python, most of which is identifier bookkeeping.

The decomposition

The tool is **iPBT**, and its contribution is the split, not the prompting.

Phase I — UI semantic grounding. Fully automated context extraction, then multimodal annotation. *Page information:* app name obtained by statically parsing `AndroidManifest.xml` (the paper’s example is that a name like SimpleNote already signals a note-taking domain); activity name from the runtime layout file; page screenshot captured over ADB. Because the annotator handles **one widget at a time**, the target widget is drawn with a **red bounding box** on the full-page screenshot so the reference is unambiguous. *Widget information:* the cropped widget image, plus four attributes from the view hierarchy — `text`, `resource_id`, `content_description`, `class`.

A multimodal model (GPT-4o mini in the evaluation) then emits two fields per widget: a **semantic label** and a **functionality** description. The worked example is an AnkiDroid `Previewer` widget whose raw attributes are

```
{"text": "Test", "resource_id": "qa", "content_description": "", "class": "android.view.View"}
```

— nearly information-free — annotated as “Question text display” / “Displays the question text to the user for review.” The output artefact is the **enriched widget context**: raw attributes plus inferred semantics.

Phase II — executable property synthesis. The user writes a Hoare-style description aligned with Gherkin Given-When-Then, in two segments: a precondition, and a function body that carries both the interaction and its expected effect. A second model receives a six-block prompt — role assignment, the target framework’s API list, the enriched widget context, two few-shot demonstrations, the property description, and output constraints — and emits the property.

The property itself is the classic triple, which the paper states inline in prose rather than as a numbered definition:

$$\langle P, I, Q \rangle$$

where P is the precondition defining when the property can be checked, I the interaction scenario defining the user actions that execute the target functionality, and Q the postcondition specifying the expected UI state afterwards. (*Unnumbered in the original — the paper contains no numbered definitions, theorems or equations anywhere in its 24 pages; its own shorthand for the structure is “Pre-I-Post”.*)

Only the API-list block is framework-specific, which is what makes the port possible: swapping that block plus two demonstrations retargets the whole pipeline from Kea (Python) to ChimpCheck (Scala).

What the numbers say

Scale. 160 property descriptions across 20 apps — 124 inherited from the Kea benchmark across 8 open-source apps, each derived from a distinct real-world historical bug, plus 36 newly authored across 12 further apps. The 12 include Firefox and WordPress plus ten closed-source Google Play apps sampled by a stated three-stage protocol (ten categories → top-20 per category → uniform random sample of one), among them Google Keep, Pinterest and Gallery at 1 B+ installs each.

RQ1 — correctness. $152/160 = 95.0\%$ for both GPT-4o and DeepSeek-V3. Ablating the functionality annotations drops this to $123/160 = 76.9\%$ and $127/160 = 79.4\%$ respectively — **18.1 and 15.6 percentage points**. All eight residual failures manifested as incorrect widget identifiers, i.e. exactly the grounding problem the first phase exists to solve, not a code-generation problem.

The demonstration sweep. On the 124-property subset: at $k = 0$, **all 124 outputs failed to preserve the required Pre-I-Post structure** — total collapse, not degradation. $k = 1$ gives 82.3 %; $k = 2$ gives 95.2 %; $k = 3, 4, 5$ show no further improvement. Two demonstrations were adopted on this evidence rather than by assertion.

RQ2 — human effort. A within-subject study, 10 graduate participants with ≥ 4 years’ programming experience, 60 tasks, VS Code with auto-completion disabled, ~45-minute warm-up. Writing an English description and running iPBT took **272.7 s** on average against **625.7 s** to write the executable property by hand — a **56 % mean reduction**, rising to **72 %** on the lowest-complexity property. Correctness went *up* as well: 29 correct via iPBT against 26 manual, with the single iPBT-path error traced to a user typo (a long-click written as a click) and the manual errors traced to unfamiliarity with the framework’s API. Inter-rater agreement $\kappa = 0.96$. As a proxy for the effort gap, the 124 descriptions average 211.3 characters against 555.0 for the corresponding executable properties.

RQ3 — robustness. 1,520 paraphrases (10 per correct description, each selected from 100 candidates generated by a *different* model, Llama-3.1-405B, greedily diversified to minimise Self-BLEU):

$$\arg\min_{S \subseteq C, |\text{S}| = k} \frac{1}{|\text{S}| \cdot (|\text{S}| - 1)} \sum_{x, y \in S, x \neq y} \text{BLEU}(x, y)$$

(Also unnumbered in the original; C is the candidate pool and $k = 10$.) Accuracy holds at **88.2 %** and **87.8 %**, against **71.7 %** and **71.9 %** ablated — the same ~16-point gap. The two models agree on 81.6 % of cases and both fail on only 5.6 %.

The failure taxonomy is the most reusable output. Across 180 and 185 failures respectively: **widget mismatch 51.7 % / 51.9 %**, logic incompleteness 23.3 % / 21.6 %, semantic deviation 18.9 % / 20.5 %, logic redundancy 6.1 % / 5.9 %. The distributions are nearly identical across two very different models, which the authors use as evidence that the taxonomy is a property of the *task* rather than of either model.

Cost. Building the enriched widget context is a one-time per-app charge amortised across all that app’s properties: 13.4 M tokens, **USD 2.00**. Per-property generation is **USD 0.044** (GPT-4o) or **USD 0.010** (DeepSeek-V3).

Generality. Ported to ChimpCheck by replacing only the API block and the two demonstrations. Of the 160 properties, **45 fall inside ChimpCheck’s expressiveness and iPBT generated 45/45 correctly**; the other 115 require DSL features ChimpCheck lacks, such as reading widget text. Claim, demonstration, and bound, in that order.

Where it stops

The authors list five threats and mitigate each: property coverage (expanded from 124 to 160 across 20 apps, with the caveat that dynamic exploration may not capture all widgets — “a common challenge of input generation in app testing”); description authenticity (mitigated by the RQ2 user-written descriptions and the RQ3 paraphrases); LLM coverage (two models only); participant representativeness; and data contamination (both models predate the Kea dataset’s release, descriptions were never public, and a third model did the paraphrasing).

The participant threat is handled with a move worth noting: the study used graduate students rather than non-technical testers *because the manual baseline requires writing code*, so the sample is biased against iPBT and the reported 56 % “likely understates the benefit for less-technical users.”

Two limits are stated outside the threats section and matter more for adoption. Context-construction cost grows with the number of distinct screens, so large industrial apps would need state deduplication or reuse of existing UI specifications. And the approach inherits its target DSL’s expressiveness ceiling entirely — the 115 ChimpCheck-inexpressible properties are not an iPBT failure and not fixable by iPBT.

Note also what this paper does *not* claim: no new bugs are reported. The 124 historical bugs are an oracle for reproduction, not a finding. The metric throughout is property-synthesis accuracy.

“Our evaluation shows that iPBT achieves 95.0% accuracy on original property descriptions”

1.2 · MODEL-BASED TESTING · 59 bugs in WeChat Pay from 208 properties nobody wrote — extracted from the acceptance-test model instead 

Model-Guided Property-Based Testing of WeChat Pay at Billion-User Scale

Authors: Xiangchen Shen, Yiting Wang, Ting Su, Jingjing Liang, Xixian Liang, Haiying Sun, Jingling Sun, Xinjie Xu, Haochuan Lu, Yuetang Deng, Pengcheng Wang, Geguang Pu, Zhendong Su, John Hughes

Venue: ASE '26 — Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering, October 12–16 2026, Munich, Germany. ACM, 13 pages. Received 2026-05-01; accepted 2026-07-01.

Links. [ACM DOI 10.1145/3832783.3834537](https://doi.org/10.1145/3832783.3834537) · [author-hosted PDF](#). **License:** CC BY-NC-ND 4.0 — the front matter states the work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License. Full text read (13 pp.). No figures are reproduced here: the NoDerivatives clause forbids it. Datasets and source are withheld for commercial reasons.

The byline is worth a second look. Three institutions plus Tencent, plus ETH Zurich, plus **John Hughes** — one of the authors of QuickCheck. This is a PBT-lineage paper that says so on its cover.

The bottleneck being named

WeChat Pay's QA organisation already runs an industrial user-acceptance-testing (UAT) system: a real-world model-based testing deployment. Use cases are written in restricted natural language; each is converted into a finite-state-machine-like model where nodes are app pages, edges are transitions triggered by user operations, and business rules attach to nodes. Per page, one operation sequence is generated from the initial page; the test fails if the reached page violates its business rules or the app crashes.

The domain is large — the paper reports **about 120 core payment scenarios associated with over 10,000 business rules, undergoing bi-weekly updates** — and the system is not obviously broken. The paper's job is to show that it is nonetheless blind in a specific, characterisable way, and it does so with two numbered limitations grounded in one running example.

Limitation-1, insufficient coverage. To avoid state-space explosion, UAT generates exactly one test per page per business rule and discards alternative routes to the same page. Three consequences: combinations of business rules across different pages are never tested together; cycle paths are executed once ($a \rightarrow b$ but never $a \rightarrow b \rightarrow a \rightarrow b$); and backward navigation is never generated ($a \rightarrow d \rightarrow f \rightarrow g$ but not $a \rightarrow d \rightarrow f \rightarrow g \rightarrow a$). Each gap is tied to a bug actually found later — the missing rule-combination produced an unexpected reading 'BaseResponse' of undefined, and clicking back within a page produced an abnormal transaction termination.

Limitation-2, oracles only where examples exist. UAT test cases are example-based. For any sequence outside the predefined set there is simply no oracle. The canonical case: UAT knows that five failed password attempts should produce a ten-minute retry prompt, but for a newly generated sequence that leaves and re-enters the page first, it cannot determine whether the failure counter was reset, so it cannot validate the result.

Plain property-based testing does not rescue this either. Random generation cannot navigate complex business logic — in the evaluation it plateaus almost immediately at roughly 15 states, trapped behind blocking pages such as password verification.

The two observations that drive the design are stated plainly: UAT test cases reach all app states because the model is comprehensive; and WeChat Pay exhibits invariant behaviours from which generic properties can be synthesised. The tool is **UAT++**, built into the existing system.

The formal core

The paper is unusually formal for an industrial experience report — nine numbered definitions and four numbered equations. A use case is a transition-based model (Definition 6):

$$u = (\mathcal{S}, \mathcal{O}, \delta, s_0, \mathcal{T})$$

with $\mathcal{S} \subseteq \mathcal{P}$ a finite set of pages, $\mathcal{O}$ a finite set of Operator APIs representing transitions, $\delta : \mathcal{S} \times \mathcal{O} \rightarrow \mathcal{S}$ the transition function, s_0 the initial page and $\mathcal{T}$ the terminal pages.

An app property (Definition 7) is $\phi = \langle Pre, I, Post \rangle$, and its semantics is Equation (1):

$$L_0 \models Pre \wedge L_n = E(L_0) \Rightarrow L_n \models Post$$

The load-bearing definition is the **page-transition invariant**, Equation (2). Given an initial page P_{init} and event trace E , the invariant $P_{target} = E(P_{init})$ holds iff:

$$\forall L \in \mathcal{I}(P_{init}), \exists! P_{target} \in \mathcal{P} : E(L) \in \mathcal{I}(P_{target})$$

where $\exists!$ is uniqueness quantification and $\mathcal{I}(P)$ is the instantiation set of page P . That uniqueness requirement is the entire trick. Only a transition that is deterministic across every explored context can serve as a reliable oracle; if E could land on different pages depending on hidden state, the transition is not an invariant and must be discarded. The property synthesis is therefore *sound by construction* relative to the manually validated use cases — a deviation at runtime is a real bug, not a guess.

The pipeline

Property synthesis. Algorithm 1, *Page-Transition Invariants Extraction*, explores every execution path in a use case up to N Operator-API steps, recording for each (P_{init}, E) pair the set of reachable pages; a pair contributes an invariant **only when that set is a singleton**. Turning a logical page into a runtime predicate needs a way to recognise it, which the paper solves with **state-independent widgets** IW derived two ways: statically, by parsing the `waitUI(target_widgets)` arguments that Operator APIs already call (giving IW_s , admittedly incomplete because developers specify only the minimum they needed), and dynamically, by capturing UI hierarchies before every Operator API across all UAT test cases — which have complete state coverage — and intersecting them per page to get IW_d . The union $IW = IW_s \cup IW_d$ identifies the page.

Exploration. Each iteration replays a predefined UAT test case to drive the app into a state random exploration cannot reach, then switches to a loop that alternates event generation with property validation. If the current layout matches a known **blocking layout** — a password prompt, say — the engine injects a domain-specific input event drawn from the payment model to get past it; otherwise it generates a random GUI event. Whenever the current layout satisfies some property's precondition, exploration is **interrupted with 50 % probability** to execute the interaction and assert the postcondition, balancing validation against new state discovery.

Shrinking. Failure traces are reduced greedily by two operators. Operator-API cycle shrinking (Equation 3) excises the span between two invocations of the same API, since their prerequisite layouts must be equivalent:

$$E' = E \ominus [op_i, \dots, e_{j-1}] \quad \text{where } op_i = op_j$$

Widget cycle shrinking (Equation 4) excises the span between the first appearance of a target widget and the event that interacts with it:

$$E' = E \ominus [e_k, \dots, e_{i-1}] \quad \text{where } w_{target} \in L_{k-1} \cap L_{i-1}$$

API cycles are prioritised; widget cycles are the fallback. Each reduction is validated by dynamic replay and discarded if it stops reproducing the bug.

The detail worth stealing is Definition 8. GUI replay on real devices is non-deterministic — asynchronous updates mean a widget present in one run may be absent in the next — so the elimination operator $\ominus$ is defined in terms of `trunc`, which returns the longest *executable* prefix of a trace under a given execution. The theory is built around the flakiness rather than assuming it away.

What the numbers say

Deployment. Four core use cases (business payment, social payment, face-to-face payment, WeChat transfer) yielded **208 properties** from single-step invariants. Implementation is roughly **15,000 lines of Go** and **6,600 lines of Python**, running in the continuous testing pipeline across Android, iOS and HarmonyOS on a cluster of **2,000 Android 12.0 devices** against daily internal builds. One testing round executes all **4,610** existing UAT test cases with **0.5 machine hours** allocated to each for property testing; ten rounds consumed approximately **20,000 machine hours**.

Bugs. **59 previously unknown bugs — 48 logic and 11 crash — of which 55 confirmed and 45 fixed.** Neither UAT nor classic PBT detected any of them. By category: unexpected page transitions 17, abnormal transaction terminations 13, crashes 11, business logic violations 9, UI rendering anomalies 9. Per use case, business payment contributed 64 properties and 35 bugs; social payment 79 and 15; face-to-face 21 and 6; transfer 44 and 3.

Coverage. Over 120 hours per approach: UAT++ and UAT both reach complete state coverage, discovering all **105 states**, while classic PBT plateaus at roughly **15**. State *transition* coverage is where they separate — UAT++ exceeds **420 transitions**, UAT reaches about **230** before plateauing near the 50-hour mark, and classic PBT manages about **60**. The paper computes these as **82.6 %** and **600 %** improvements, and adds a footnote that the curves show no plateau, so 420 is a lower bound.

Shrinking. On 20 randomly selected bugs: **79.9 % average reduction**, maximum 95.2 %, minimum 50.0 %, median 86.1 %. Traces fall from an average of 41.5 events to 5.1 in an average of 12.3 iterations. Iterations rather than wall-clock are reported “because GUI event replay is inherently non-deterministic and subject to device-specific overhead.”

Economics. The comparison management will read: developing the UAT suite required **2,000 person-hours to deliver 1,000 test cases** and yields about **10 bugs monthly**; UAT++ found 59 over roughly 20,000 machine hours with no additional manual scripting.

Extension. Two-step transition invariants generate **809 properties** and found one bug in a single round that single-step invariants missed — evidence the parameter has headroom, and that the cost/comprehensiveness trade-off is a deployment decision rather than a fixed constant.

Where it stops

External validity is the only threat category discussed: the evaluation is one app, and other payment apps' architectures may differ. The mitigation argument is reasonable — property synthesis reads the use-case model, and exploration and shrinking operate on event traces, so nothing depends on app-specific logic; another team “only need to provide an equivalent model as input.” No internal or construct validity discussion appears, RQ2 is single-run with no variance reported, and RQ3 samples 20 bugs. There is no artifact.

The most valuable section is the honest gap between theory and deployment. In principle UAT++ has zero false positives, because properties come from manually validated use cases. In practice three engineering-level sources appeared: asynchronous rendering latency causing premature postcondition checks, fixed by detecting loading widgets by classname or injecting waits only where needed; flaky Operator APIs, whose failures both produce spurious violations and corrupt the dynamic *IW* capture, fixed by monitoring API status and falling back to random exploration on failure while discarding the bad snapshot; and ambiguous page identification where distinct pages share identical *IW*, handled by whitelisting.

The authors also mark their own scope honestly. Properties currently cover UI navigation only; extending to data-level invariants — arithmetic correctness of transaction computations, consistency across business entities, “akin to the invariants detected by Daikon” — is named as future work rather than implied to be already covered.

“It has successfully discovered 59 previously unknown bugs that evaded the existing UAT system.”

1.3 · DBMS FUZZING · If a SQL feature isn't in the seed corpus, mutation can't invent it — 61 bugs and up to 12.5× branch coverage [👍😊👉](#)

SmartFuzz: Leveraging Large Language Models and Feature Composition to Generate High-Quality Seeds for Database Fuzzing

Authors: Li Lin, Jintai Hong, Yanlin Zhuang, Rongxin Wu

Venue: *Proc. ACM Program. Lang.* 10, OOPSLA2, Article 344 (October 2026), 29 pages. All authors: School of Informatics, Xiamen University.

Links. [ACM DOI 10.1145/3839476](https://doi.org/10.1145/3839476) · [author-hosted PDF](#). **License: CC BY 4.0** — the front matter states the work is licensed under a Creative Commons Attribution 4.0 International License. Full text read (29 pp.).

The bottleneck being named

Mutation-based DBMS fuzzing has been iterating on mutation operators for years. This paper argues the binding constraint is upstream, in seed selection, and it makes the argument structurally rather than statistically.

The claim, from the motivating section: a particular MonetDB bug is hard for existing fuzzers to find “not because of insufficient mutation capability” but because the required feature — a specific function and its usage patterns — is absent from their initial seed corpora. When a feature is missing from the pool, insert/delete/replace mutation cannot synthesise it, so the corresponding paths are **unreachable regardless of fuzzing duration**. That is a reachability claim. “Run it longer” is not an answer to it.

Two supporting problems are then measured rather than asserted. **Provenance:** Table 1 surveys eight mutation-based DBMS fuzzing papers published since 2020 and finds seven of eight draw seeds from official unit or regression test suites — which are laborious to maintain and simply do not exist for newer or less-studied systems. **Distribution:** those suites are built for functional correctness verification, so they systematically under-represent rare query patterns and dialect-specific features; cross-DBMS transfer inherits the same bias plus a dialect mismatch. And prompting an LLM directly does not fix it either, since — as Fuzz4All observed — models regress toward training-distribution SQL and produce structurally homogeneous output, precisely the wrong failure mode for fuzzing.

Each of the two seed strategies gets its own motivating bug with a real tracker ID. Documentation-derived: MonetDB issue #7709, assigned **CVE-2025-69761**, a syntactically valid `field()` call over a subquery that trips `Assertion 's' failed` — well-defined SQL, absent from every existing corpus. Crash-derived: DuckDB issue #20615, an ASan wild-pointer read, produced by extracting the `LAG` crash feature from a `SQLite` ticket and recomposing it with an `OVER()` window specification. A cross-DBMS feature transplant, shown rather than described.

The pipeline

Three stages, decomposed on what the paper calls the task-decomposition principle of agentic workflows — explicitly instead of one monolithic prompt.

Stage 1 — feature extraction into a knowledge base. A **rule-based** document parser (not an LLM) walks official SQL reference pages and extracts, per feature, up to three components: the grammar rule or specification, the natural-language description of semantics and constraints, and example snippets. Where the documentation omits explanation or examples, an LLM synthesises the missing piece — a deliberate move to preserve sparsely documented features and “reduce bias toward well-documented constructs,” which is an anti-Matthew-effect design choice worth naming. In parallel, a **crash analyzer** mines historical crash-triggering SQL from eight DBMS bug repositories. Critically it does *not* treat a crashing input as an atomic seed, on the grounds that raw crash inputs are highly coupled to particular configurations; it decomposes them into reusable fine-grained features instead.

That decomposition is Algorithm 1, *Crash Feature Extraction*, in three steps. **Reduce** the crashing input to F_{min} by SQL-aware delta debugging. **Generate passing variants** by applying minimally-perturbing mutation rules across four levels — identifier (constant substitution, `COLLATE` removal, charset switches), operator (division vs integer division, equality vs null-aware comparison, precedence), structure (parenthesis removal, subquery flattening, `UNION` → `UNION ALL`), and clause/statement (`HAVING`, window functions, `LATERAL` joins, `LIMIT/OFFSET`) — retaining any variant that still executes but no longer crashes. **Abstract** by AST-level diffing of F_{min} against each passing variant, keeping the nodes whose change coincides with the crashing-to-passing transition, together with their surrounding syntactic context, then **normalising query-specific identifiers and literals while preserving semantic categories and structural relationships**. That normalisation is what makes a `SQLite` crash feature reusable in DuckDB.

Stage 2 — seed synthesis. “Feature composition” is not a learned operation; it is a scheduling policy plus a hard prompt constraint that all supplied features must co-occur and interact in one SQL program. *How many* is set by a three-stage combination policy with a stated budget: **30 % single-feature** prompts as a cold start establishing validity, **50 % two-feature** prompts as the main exploration phase, **20 % three-feature** prompts, rate-limited because higher-order composition degrades synthesis validity. *Which* is set by coverage-driven adaptation: **after every 100 synthesised seeds**, features are sorted

ascending by the sum of their prompt-usage count and their valid-seed count, and the next round samples preferentially from the **lowest-ranked 30 %**, with repeatedly used pairs temporarily deprioritised.

The prompt itself has four blocks — a system role declaring a fuzzing-oriented SQL synthesis agent (explicitly to steer away from generic text-to-SQL), a goal, the feature records, and five numbered synthesis constraints covering cross-feature composition, semantic stress (implicit casting, NULL propagation, precedence, evaluation order), structural complexity, validity and diversity, and strict JSON output. Note that **crash-derived features are typed as a first-class feature kind alongside operators and clauses** — that typing is what lets the composer mix a crash pattern with a documentation feature in one statement. Prompts carry no conversational state, so they are issued and evaluated in parallel. A lightweight filter checks syntactic correctness and basic executability; failures are discarded with no repair loop.

Stage 3 — compatibility-aware mutation. The classic problem: off-the-shelf mutators support a fixed grammar subset, so dialect-specific fragments break IR construction and block mutation entirely. The fix is a mark–mutate–restore workflow. A fragment marker runs the parser in **error-recovery mode**, recording unrecognised token ranges while continuing to parse, and comments them out; the mutator works on the parsable remainder; a restorer uncomments. The paper is emphatic that this identification is “fully rule-based and does not involve an LLM.” The four-line worked example shows a PostgreSQL `INET` column surviving the addition of `NOT NULL`. The payoff is that SmartFuzz is a plug-and-play seed generator requiring no changes to the host fuzzer’s mutation engine.

On formalism. The paper contains no numbered definitions, theorems or equations; the only arithmetic it states is a table note defining coverage gain as $(\text{SmartFuzz Cov.} - \text{Baseline Cov.}) / \text{Baseline Cov.}$. Its contribution is empirical and engineering, and it does not dress that up as theory — which is the right call, and worth contrasting with papers that manufacture a Definition 1 to look rigorous.

What the numbers say

Subjects. Four open-source relational systems, deliberately spanning the popularity range: MonetDB v11.54.0 (449 GitHub stars, DB-Engines rank 153), Virtuoso v7.2.17 (6.1 K, rank 91), DuckDB v1.4.1 (35.3 K, rank 42), ClickHouse v23.5.3.4 (45.1 K, rank 30).

Bugs. 61 previously unknown bugs — MonetDB 32, Virtuoso 20, DuckDB 8, ClickHouse 1 — with 60 confirmed and 60 fixed at submission and one awaiting response. One CVE, **CVE-2025-69761**. By type: 31 assertion failures, 9 segmentation violations, 9 abort, 8 buffer overflows (6 heap, 2 stack), 3 null-pointer dereferences, 1 double free.

Coverage against seed-selection baselines. All configurations use the *same* host fuzzer and differ only in seed source, which is the right controlled comparison. Branch-coverage improvement over 24 hours per DBMS: **+260.0 % / +211.1 % / +256.0 % / +1250.0 %** over the host fuzzer’s default seeds (MonetDB / Virtuoso / DuckDB / ClickHouse respectively); **+50.0 % / +27.1 % / +134.8 %** over official local test suites on the three systems that have them; and **+38.5 % / +51.4 % / +50.8 % / +300.0 %** over a re-implemented cross-DBMS seed-transfer baseline. Reported as significant at $p \leq 0.05$, over five repetitions per configuration. The headline “up to 12.5×” is the ClickHouse figure. On bugs, SmartFuzz **subsumes every bug found by every baseline and finds 46 more**.

The mechanism check. Two measurements do more work than the coverage table. First, the feature-coverage gap: SmartFuzz seeds exercise **+360 / +723 / +519 / +1,231** additional documented DBMS

features over the host fuzzer’s defaults. Second, and better: of the 61 bugs, **45 were triggered by dialect-specific or rare features against 16 by standard SQL, and 27 bug-triggering feature sets are not covered by any existing seed corpus at all**. That last number is the reachability claim from the introduction, measured.

Ablations. Removing documentation features or crash features each degrades results while still beating traditional seed selection. In a crash-only configuration, extracting and recomposing crash *features* rather than replaying translated crash SQL directly improves coverage by 7.9 % / 14.5 % / 29.6 % / 7.9 % and bug counts from 11 → 18, 5 → 11, 2 → 8, 0 → 0. Against an LLM-only configuration with no coverage-guided mutation, SmartFuzz finds **57 additional bugs** — which answers the obvious “is the model doing all the work?” with a table rather than a paragraph.

Against other paradigms. Generation-based fuzzers (a grammar-based tool and its extension) find **5 bugs in total** across the four systems against SmartFuzz’s 61 under the same 24-hour budget; an LLM-based general-purpose fuzzer finds **2**. The authors immediately disclaim these as not fully controlled comparisons — see below.

Adoption cost. The implementation is **2,128 lines of Python and 112,522 lines of C++, of which 1,914 are new and 110,608 inherited from the host fuzzer**. Porting to a new DBMS “required roughly eight hours” for one author: extend the parser for dialect syntax, implement a client interface. Against the cited figure that supporting selected SQLite features in a grammar-based generator took 10,020 lines of code, that is the whole practicality argument in two sentences.

Model dependence. GPT-4o-mini is the main model, chosen for cost-effectiveness. A nine-model study across three closed and six open models reports syntactic validity from 62.0 % to 94.5 % and semantic validity from 55.0 % to 92.1 %, with the frontier models a few points above GPT-4o-mini’s 88–90 % / 84–87 % and the small open models 20+ points below. The technique degrades gracefully rather than depending on one vendor.

Where it stops

Internal validity: one baseline was re-implemented from its paper because the original is not public, mitigated by hand-written tests against the reported semantics. And the cross-paradigm comparison is disclaimed in the authors’ own words as “not a fully apples-to-apples component comparison” — SmartFuzz uses an LLM for seed synthesis while the generation-based tools use manually encoded grammars and no LLM at all — with the results to “be interpreted as demonstrating the practical benefit of combining LLM-guided seed synthesis with coverage-guided mutation, rather than proving that mutation-based fuzzing is universally superior.” SmartFuzz is positioned as complementary. Conceding your own unfair comparison before a reviewer finds it is the strongest move in the paper’s discussion.

External validity: four open-source *relational* systems only; proprietary and NoSQL systems may differ, and adapting would require extending the parsing and mutation infrastructure, which is relational by design.

Three gaps the paper does not acknowledge are worth noting for anyone re-quoting it. Repetition count is stated for the coverage RQ but not for the bug-finding RQ or the discussion tables. No total seed count, token count, or LLM wall-clock cost is reported anywhere, which leaves the “favourable cost-effectiveness trade-off” rationale for the model choice unquantified. And the documentation gap-filling step — where an LLM writes explanations the official docs omit — is never validated for fidelity to actual DBMS behaviour, which is a live hallucination surface in an otherwise carefully rule-based parser.

“SmartFuzz improves both coverage and bug discovery across 4 DBMSs including MonetDB, Virtuoso, DuckDB, ClickHouse, uncovering 61 previously unknown bugs.”

Cross-Paper Synthesis

The specification is the bottleneck, and it has been for a while. Three papers, three domains, one shape. PBT frameworks solved execution and input generation and left property authoring to a human. Mutation fuzzers solved mutation operators and left seed provenance to whoever maintains the regression suite. Acceptance testing solved model construction and left oracles to whoever wrote the example. In each case the automated part got good enough that the manual part became the ceiling — and in each case the field kept optimising the automated part for several more years before anyone measured the ceiling. The 2026 move is to stop.

None of them asks a model to invent the specification. This is the substantive agreement and it is easy to miss under the shared “we used an LLM” surface. iPBT’s model translates a human’s English into a framework’s API, and its first phase does no synthesis at all — it annotates widgets so the second phase has something to refer to. SmartFuzz’s model composes features mined by a *rule-based* parser from official documentation, and the paper says twice, unprompted, that the fragment identification “is fully rule-based and does not involve an LLM.” UAT++ uses no model whatsoever: its properties come from a graph traversal over a hand-validated use-case model, filtered by a uniqueness test. The generative capability is deployed for transcription between representations, never for authoring content that has to be correct. SmartFuzz makes the reason explicit by citing Fuzz4All’s finding that models regress to training-distribution SQL — the exact behaviour you do not want when the goal is distributional novelty.

Three different answers to “how do I know the generated spec is right?” This is where they diverge, and the divergence is instructive. UAT++ gets soundness by construction: keep only transitions whose target page is unique across all explored paths, so a deviation is definitionally a bug, and the paper’s theoretical false-positive rate is zero. SmartFuzz sidesteps the question — a bad seed is merely wasted budget, so a lightweight syntax-and-executability filter suffices and rejected candidates are dropped without a repair loop. iPBT has neither option, because a wrong property is a silent wrong oracle, and pays for it: 5 % residual error on original descriptions and 12 % on paraphrases, with no automatic detection. The ranking is not about technique quality; it is about whether the domain admits a cheap total check. Where it does, admit the model freely. Where it does not, the residual error rate is your result and you should report it, as iPBT does.

Mining what the organisation already owns. The genuinely transferable insight is where the specifications came from. WeChat Pay’s oracles were latent in an acceptance-test model built for a different purpose. SmartFuzz’s seeds came from SQL reference documentation and from crash reports that every project already files. iPBT’s widget semantics came from screenshots and view hierarchies that the runtime emits for free. Not one of these artefacts was created for testing, and not one was being mined. The question worth carrying into any project — *what specification is latent in an artefact we already maintain?* — is more portable than any of the three techniques.

Decomposition into extraction plus composition. All three separate a knowledge-building phase from a generation phase, and in all three the ablation shows the knowledge phase is where the value is. iPBT’s annotation phase is worth 18.1 points. SmartFuzz’s crash-feature abstraction — decomposing a crash into reusable normalised features rather than replaying it — is worth up to 4× the bug count in the crash-

only configuration. UAT++’s *IW* derivation, static plus dynamic intersection, is what makes any property checkable at all. The architectural lesson generalises: when a pipeline’s second stage is a general-purpose model, invest in the first stage, because that is the part that is specific to your problem and therefore the only part the model cannot supply.

The cost/economics table has arrived and is doing real work. iPBT reports USD 2.00 per app for context and USD 0.044 per property. UAT++ compares 2,000 person-hours for 1,000 test cases yielding 10 bugs monthly against 59 bugs over 20,000 machine hours. SmartFuzz reports eight hours to port to a new DBMS against a cited 10,020 lines of code for the grammar-based alternative. Three papers, three different cost units, all placed in the reader’s decision path rather than in an implementation footnote. Note the asymmetry though: SmartFuzz reports adoption cost but never its own token or dollar spend, which is the one gap in an otherwise thorough accounting.

Where the three are one experiment apart. SmartFuzz’s Table 5 asks, for every bug found, whether the triggering feature was present in the existing seed corpus — and answers “no” for 27 of 61. That column converts a bug count into a causal argument, and neither of the other two papers has its equivalent. UAT++ reports 59 bugs but not how many were reachable in principle under the old one-test-per-page policy. iPBT reports 95.0 % accuracy but not which property *classes* the residual 5 % belongs to beyond “widget identifiers.” The reusable instrument here is a per-finding column stating what the baseline could not have done, and it costs one table.

Writing & Rationale Insights

Turn your claim from probabilistic into structural, and reviewers lose their standard objection.

SmartFuzz could have written “our seeds find more bugs.” It wrote that a feature absent from the corpus makes its paths unreachable “regardless of fuzzing duration.” A quantitative claim invites “did you run the baseline long enough?”; a reachability claim does not admit that response at all. If your contribution can be stated as *this class of outcome is impossible without our component*, spend the paragraph — it converts an incremental result into a categorical one at zero experimental cost.

A total-collapse ablation beats a graded curve for proving necessity. iPBT’s demonstration sweep reports that at $k = 0$, all 124 outputs failed to preserve the required structure. Not “accuracy declined” — every single one failed. A cheap experiment that produces a 0 % is disproportionately persuasive, because it establishes that a component is *necessary* rather than merely helpful, which is a different logical claim. Run the degenerate configuration even when you are sure it will fail; especially then.

Ground each named limitation in a bug you later found. UAT++ states two numbered limitations of the incumbent system, and each is tied to a specific defect in the evaluation — the missing rule-combination produced a concrete undefined-property error, the missing back-navigation produced an abnormal transaction termination. This welds motivation to results, so a reader arriving at the bug table has already been told why those bugs exist. The alternative, where §1 asserts a gap and §5 reports numbers with no thread between them, is the default and it is much weaker.

State the clean theoretical claim, then itemise every way practice violated it. UAT++ says its false-positive rate is zero by construction, and then lists three engineering-level sources of false positives found in deployment — rendering latency, flaky operator APIs, ambiguous page identification — each with the mitigation shipped. This is the single most credible page in the paper. A reviewer who was preparing to attack the zero-FP claim finds the attack already made, already answered, and already counted as a contribution.

Concede your own unfair comparison, in your own words, before anyone else does. SmartFuzz’s discussion says outright that comparing an LLM-based pipeline against manually encoded grammars is “not a fully apples-to-apples component comparison,” and reframes the result as complementary rather than superior. This costs a sentence and removes a reject reason. The general form: find the strongest methodological objection to your own headline table and put it in the paper, phrased as positioning rather than apology.

Flip a sampling limitation into a lower bound. iPBT’s user study used graduate students rather than non-technical testers, which sounds like a weakness. The paper points out that the manual baseline *requires* coding ability, so the sample is biased against the tool, and therefore the 56 % time reduction “likely understates the benefit.” Same fact, opposite valence, and honest — the conclusion actually follows. Look for threats that constrain your result in the conservative direction and say so explicitly, because readers do not compute this for you.

Replace “prior work does X” with a table of prior work doing X. SmartFuzz’s related-work claim is that existing DBMS fuzzers seed from official test suites. Instead of asserting it, eight rows: paper, venue, seed source, seven of eight confirming. This is very high argument density per column-inch, it is falsifiable, and it makes the gap a measured fact rather than a rhetorical setup. Any paper whose motivation is “everyone does X” should be able to produce this table, and a paper that cannot should wonder whether everyone does.

Publish the near-null results next to the spectacular ones. SmartFuzz reports +1.8 % and +1.3 % on Virtuoso in the same tables as +248 % and +340 % on DuckDB, unhidden and unexplained-away. Leaving the weak cells in is what makes the +1,250 % cell believable. A results table with no unimpressive numbers reads as selected, whatever its actual provenance.

Say where you did *not* use the model. In an LLM paper, “this identification is fully rule-based and does not involve an LLM” is worth more than another accuracy point. Explicit negative design decisions calibrate a reader’s trust in the positive ones, because they demonstrate that the boundary was drawn deliberately rather than by default. UAT++ does the same thing implicitly by using no model at all where a model was available.

Give every heuristic a number. SmartFuzz writes 30 % / 50 % / 20 % for its combination budget, “after every 100 synthesised seeds” for recalibration, and “lowest-ranked 30 %” for sampling. Most papers would write “we adaptively prioritise under-explored features,” which is unreproducible and tells a reader nothing about whether the heuristic is tuned or arbitrary. The numbers cost nothing and are the difference between a described system and a reproducible one.

Promote the messy reality into the formalism instead of the threats section. UAT++ needs GUI replay to be non-deterministic, so it defines a truncation operator returning the longest executable prefix under a given execution, and then defines its elimination operator in terms of that. The flakiness is in the definitions, not in a paragraph apologising for the definitions. When your theory has to survive contact with real hardware, building the contact into the theory is more honest and reads as more competent than idealising and disclaiming.

Carry one running example through every definition. UAT++ introduces its pages, layouts and operator APIs once and reuses them for all nine definitions, the synthesis walkthrough and the shrinking walkthrough. Across nine definitions this is the difference between a readable formal section and one nobody finishes. iPBT does the same with a single file-manager property. The cost is a little contrivance in the example; the benefit is that a reader never reloads context.